

Documento dei requisiti: BitPub

1. Introduzione e Descrizione

Cos'è BitPub BitPub è una piattaforma distribuita di ultima generazione dedicata ai Giochi Connessi, progettata specificamente per digitalizzare l'esperienza fisica all'interno di pub e sale giochi (es. calciobalilla, biliardo, freccette). Il sistema centralizza due aspetti fondamentali del gaming interattivo mediante un'architettura Edge-Cloud a Microservizi:

- 1. Edge Layer (IoT & Real-Time):** I giochi fisici inviano eventi a un broker MQTT locale (Mosquitto). Un nodo di prossimità elabora, valida e bufferizza gli eventi per gestire eventuali disconnessioni verso il cloud, fornendo reattività in tempo reale.
- 2. Cloud Layer & Front-office (Cliente/Admin):** Una flotta di microservizi (Spring Boot) riceve gli eventi, calcola i punteggi, gestisce i tornei, le leaderboard e salva i dati. I giocatori e gli amministratori interagiscono tramite una WebApp React interfacciata via API Gateway, con aggiornamenti in tempo reale via WebSocket.

L'obiettivo è unire l'esperienza reale tipica dei pub alle dinamiche degli e-sports (classifiche, tornei automatici, matchmaking), garantendo resilienza e scalabilità.

2. Requisiti funzionali di alto livello

ID	Requisito (Alto Livello)	Descrizione e Dettagli
RF-01a	Gestione Piattaforma Globale	Il sistema deve permettere ai PLATFORM_ADMIN di gestire il database centralizzato, effettuare il backfill delle statistiche globali e supervisionare l'infrastruttura Cloud a microservizi.
RF-01b	Gestione Locali e Macchine	Il sistema deve permettere ai LOCALE_ADMIN di definire l'anagrafica dei propri pub fisici e registrare nuove Macchine hardware, generando ID univoci da accoppiare ai sensori IoT o ai simulatori.
RF-02	Gestione Utenti (Gerarchico)	Il sistema deve gestire l'autenticazione via JWT con RBAC (Role-Based Access Control). Sono previsti ruoli separati: PLAYER (per giocare), LOCALE_ADMIN (per i pub), GAME_ADMIN (per i regolamenti) e PLATFORM_ADMIN.

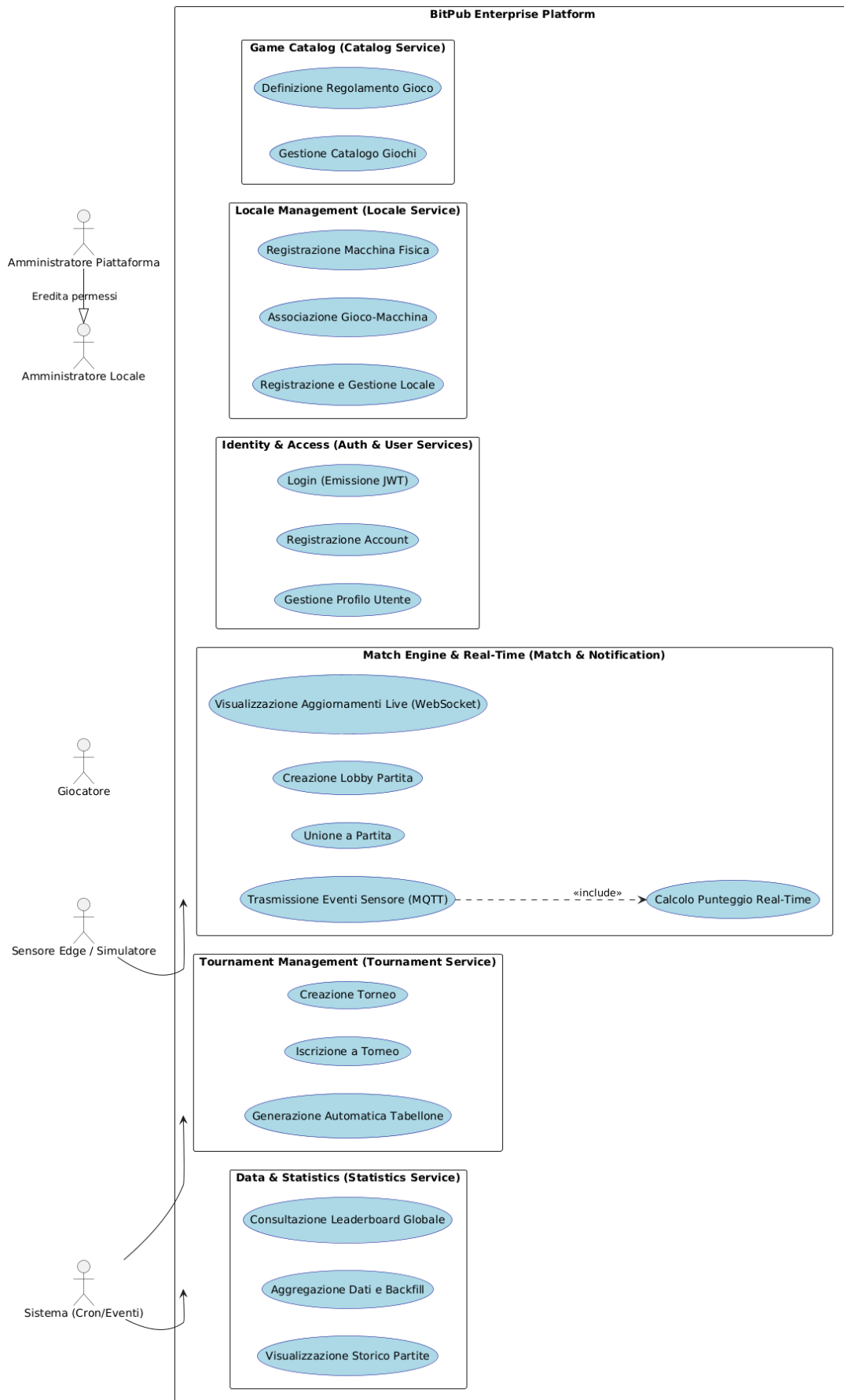
ID	Requisito (Alto Livello)	Descrizione e Dettagli
RF-03	Catalogo Giochi e Regole	Il sistema deve permettere al GAME_ADMIN di gestire il game-catalog-service, configurando i tipi di gioco (es. Calciobalilla a 10 goal, Freccette 501) e le metriche generate dai sensori fisici.
RF-04	Motore Matchmaking e Real-Time	Il sistema deve gestire l'intero ciclo di vita di una partita tramite il match-service. Dalla creazione della Lobby sulla WebApp, all'elaborazione degli input MQTT dall'Edge Node, fino all'aggiornamento WebSocket in tempo reale.
RF-05	Tornei e Generazione Tabelloni	Il sistema deve gestire tramite il tournament-service l'iscrizione dei giocatori e generare automaticamente i bracket/tabelloni a eliminazione diretta una volta raggiunto il numero massimo di iscritti.
RF-06	Leaderboard e Statistiche	Il sistema deve aggregare costantemente i risultati delle partite concluse (statistics-service) per generare classifiche globali filtrabili per utente e per tipo di gioco.

3. Requisiti non funzionali

Categoria	Requisito (Descrizione)	Criterio / Vincolo
Accessibilità e Compatibilità	Il sistema deve essere accessibile tramite una WebApp moderna (React 19) e responsiva.	La dashboard dei punteggi deve essere leggibile anche da mobile durante il gioco al tavolo.
Performance	Il sistema deve elaborare gli input fisici (sensori) e aggiornare la UI istantaneamente.	Obiettivo: Trasmissione eventi via MQTT ed emissione aggiornamenti WebSocket in < 50ms per garantire l'effetto "Real-Time".

Categoria	Requisito (Descrizione)	Criterio / Vincolo
Usabilità	Il sistema deve garantire un'alta soddisfazione e facilità di ingaggio per l'utente finale nel pub.	Criterio di Successo: Processo di avvio di una partita (dal pairing con la macchina all'inizio) completato in meno di 3 click.
Affidabilità (Disponibilità)	Il sistema deve gestire disconnessioni temporanee di rete nel pub senza perdere i punti segnati.	Criterio Critico: Il nodo BitPub-Edge deve fungere da buffer locale, garantendo la delivery "exactly-once" al ritorno della connessione.
Sicurezza	Il sistema deve proteggere API e identità degli utenti.	Rischio (Mitigazione): API protette da gateway-service tramite convalida token JWT; password hashate nel database (PostgreSQL).
Scalabilità	L'architettura cloud deve poter scalare all'aumentare dei pub e dei giocatori connessi.	Vincolo: L'intera suite di backend deve rispettare l'architettura a Microservizi dockerizzati.

4. Diagramma dei casi D'Uso



5. Descrizione dei Casi d'uso

Dettagli Caso d'Uso 1: Creazione Locale e Registrazione Macchina

Id: 1

Breve Descrizione: Consente all'Amministratore del Locale di inserire il proprio pub nel sistema e generare gli ID univoci per i tavoli da gioco fisici (Macchine).

Attori Principali: LOCALE_ADMIN, PLATFORM_ADMIN

Attori Secondari: locale-service

Precondizioni:

1. L'utente ha effettuato il login con privilegi di amministratore locale.

Flusso Principale:

2. L'amministratore accede alla sezione "Gestione Locali".
3. Inserisce i dati del pub (Nome, Indirizzo) e conferma. Il locale-service salva i dati in PostgreSQL.
4. Seleziona il locale appena creato e clicca su "Registra Macchina".
5. Definisce il nome (es. "Calciobalilla Sala Nord") e seleziona il tipo di gioco dal catalogo.
6. Il sistema genera e restituisce un ID Macchina univoco.

Postcondizioni: La macchina è creata e il suo ID è pronto per essere iniettato nel software dell'Edge Node o nel Simulatore del pub.

Dettagli Caso d'Uso 2: Configurazione Catalogo Giochi

Id: 2

Breve Descrizione: Permette di inserire un nuovo sport da pub (es. Freccette) definendone il regolamento.

Attori Principali: GAME_ADMIN

Attori Secondari: game-catalog-service

Precondizioni: Il Manager dei giochi è autenticato.

Flusso Principale:

1. Il Game Admin accede al "Catalogo Giochi" e seleziona "Aggiungi Gioco".
2. Inserisce Nome (es. Biliardo Palla 8) e le regole di chiusura match.
3. Conferma i dati.

4. Il sistema valida il payload e aggiorna il DB catalogo.

Postcondizioni: Il gioco è disponibile per essere associato a nuove macchine fisiche e a nuovi tornei.

Dettagli Caso d'Uso 3: Creazione Torneo

Id: 3

Breve Descrizione: Consente la strutturazione di un nuovo torneo ufficiale con numero di posti limitato e associazione a uno specifico tipo di gioco.

Attori Principali: PLATFORM_ADMIN

Attori Secondari: tournament-service

Flusso Principale:

1. L'admin accede alla dashboard "Tornei" e clicca "Crea Nuovo Torneo".
2. Definisce Titolo, Tipo Gioco (dal catalogo) e Numero massimo di iscritti (es. 8).
3. Salva la configurazione.

Postcondizioni: Il torneo entra nello stato "Aperto alle iscrizioni" ed è visibile ai giocatori.

Dettagli Caso d'Uso 4: Iscrizione a Torneo e Generazione Bracket

Id: 4

Breve Descrizione: I giocatori competono per un posto nel torneo; al riempimento, il sistema genera il tabellone matematico.

Attori Principali: PLAYER

Precondizioni: Esiste un torneo "Aperto".

Flusso Principale:

1. Il giocatore naviga nella lista Tornei e seleziona "Iscriviti".
2. Il sistema convalida la richiesta (verifica che i posti non siano esauriti).
3. Il giocatore viene aggiunto alla lista dei partecipanti.
4. Estensione (Generazione Automatica): Quando si raggiunge il tetto massimo di partecipanti, il tournament-service genera automaticamente i match del primo turno (bracket).

Postcondizioni: L'utente è iscritto. Se il torneo è pieno, i primi incontri vengono resi disponibili per essere giocati.

Sequenze Alternative: A3 (Torneo al completo).

Dettagli Caso d'Uso 5: Avvio Partita e Sincronizzazione Real-Time (Core)

Id: 5

Breve Descrizione: Il processo core del sistema: associazione di giocatori a una macchina hardware, ricezione degli input fisici via MQTT e aggiornamento WebSockets per gli spettatori.

Attori Principali: PLAYER

Attori Secondari: Dispositivo fisico (Edge Node / Simulatore), match-service, notification-service.

Precondizioni: La macchina hardware è accesa e connessa all'Edge Node locale.

Flusso Principale:

1. Due giocatori prenotano la macchina tramite WebApp (Lobby).
2. Lo stato della partita sul Cloud diventa IN_PROGRESS.
3. Un sensore hardware scatta (es. un goal nel calciobalilla). L'evento è inviato al broker MQTT locale.
4. L'Edge Node bufferizza e inoltra l'evento MQTT al match-service sul Cloud.
5. Il match-service elabora il punto, calcola il punteggio aggiornato e salva nel DB.
6. Il notification-service diffonde il nuovo punteggio tramite WebSockets.
7. La UI React di tutti gli utenti si aggiorna in real-time.

Postcondizioni: Il punteggio avanza finché le regole di chiusura non terminano la partita (stato FINISHED).

Sequenze Alternative: A1 (Disconnessione Cloud), A2 (Macchina già occupata).

Dettagli Caso d'Uso 6: Visualizzazione Leaderboard

Id: 6

Breve Descrizione: Aggregazione dei dati storici delle partite per costruire la classifica dei migliori giocatori.

Attori Principali: Tutti gli utenti.

Attori Secondari: statistics-service.

Flusso Principale:

1. L'utente naviga nella pagina "Classifiche".
2. Seleziona il Tipo di Gioco dal menu a tendina.
3. Il statistics-service interroga il database per vittorie, sconfitte e rapporto W/L.

4. Restituisce il rank ordinato.

Postcondizioni: L'utente consulta le proprie e le altrui performance.

6. Descrizione Scenari Alternativi (Estensioni ed Errori)

Scenario: Disconnessione Cloud (Intervento Nodo Edge)

Dettagli Scenario A1

Id: A1

Breve Descrizione: Si verifica se il pub perde momentaneamente l'accesso a internet (rete Cloud inarrivabile) durante una partita in corso (UC 5).

Attori Principali: Sistema BitPub-Edge locale

Precondizioni: Partita IN_PROGRESS, ricezione di eventi MQTT dai sensori, WAN down.

Flusso Principale:

1. Il sensore hardware registra un punto e lo passa al broker MQTT locale (Mosquitto).
2. Il Nodo Edge tenta di invocare il REST/gRPC del match-service cloud, riscontrando un timeout.
3. Il Nodo Edge non scarta l'evento, ma lo salva in un buffer locale sicuro.
4. Il Nodo Edge continua ad ascoltare e salvare altri eventi di gioco.
5. Quando la connettività si ripristina, il Nodo Edge svuota il buffer garantendo la sincronizzazione (exactly-once delivery).

Postcondizioni: Nessun dato hardware viene perso; il server cloud riallinea i punteggi ritardati e notifica i client.

Scenario: Macchina Occupata (Race Condition nella prenotazione)

Dettagli Scenario A2

Id: A2

Breve Descrizione: Si verifica durante l'UC 5 se due player cercano contemporaneamente di avviare una Lobby sulla stessa macchina hardware di un pub.

Attori Principali: PLAYER

Precondizioni: Macchina visualizzata come IDLE.

Flusso Principale:

1. Il Player A e il Player B premono "Inizia Partita" sulla medesima macchina.

2. Il sistema gestisce l'accesso concorrenziale: elabora prima la richiesta di A.
3. Lo stato passa a WAITING_FOR_PLAYERS.
4. La richiesta di B arriva e trova lo stato variato.
5. Il sistema rifiuta B con messaggio: "Attenzione: Macchina al momento occupata da un'altra lobby".

Postcondizioni: B deve selezionare un altro tavolo.

Scenario: Torneo al Completo

Dettagli Scenario A3

Id: A3

Breve Descrizione: Scenario di validazione limite iscrizioni nell'UC 4.

Attori Principali: PLAYER

Flusso Principale:

1. L'utente clicca "Iscriviti" su un torneo appena riempitosi.
2. Il tournament-service valuta la capienza (es. iscritti == maxCapacity).
3. La transazione viene rifiutata.
4. Viene mostrato il messaggio: "Le iscrizioni per questo torneo sono chiuse in quanto è stato raggiunto il limite massimo".

Postcondizioni: L'utente non viene aggiunto; il torneo procede con la chiusura delle iscrizioni.

7. Matrice di mappatura dei requisiti

Requisito	UC 1	UC 2	UC 3	UC 4	UC 5	UC 6
RF-01a (Gestione Piattaforma)			X			
RF-01b (Locali e Macchine)	X					
RF-02 (Gestione Utenti)	X	X	X	X	X	X
RF-03 (Catalogo Giochi)		X				

Requisito	UC 1	UC 2	UC 3	UC 4	UC 5	UC 6
RF-04 (Motore Matchmaking)					X	
RF-05 (Tornei e Tabelloni)			X	X		
RF-06 (Statistiche e Leaderboard)					X	X